Vulnerability Research Report

PortSwigger

Web Security Academy — PortSwigger

Type	Labs Covered	Platform	Status
SQL Injection	5	PortSwigger / WSA	All Solved

Date: April 11, 2026 | Classification: Web Application Vulnerability

Table of Contents

Lab 1	SQL Injection — WHERE Clause: Retrieving Hidden Data
Lab 2	SQL Injection — Login Bypass
Lab 3	SQL Injection UNION Attack — Determining Column Count
Lab 4	SQL Injection UNION Attack — Finding a Column Containing Text
Lab 5	SQL Injection UNION Attack — Retrieving Data from Other Tables

LAB 1

SQL Injection — WHERE Clause: Retrieving Hidden Data

LAB 01	APPRENTICE	SOLVED
SQL Injection Vulnerability in WHERE Clause		

Difficulty: Apprentice	**Target:** Product category filter	**Status:** Solved

2. Vulnerability

The application contains a SQL injection vulnerability in the product category filter. User-supplied input is passed unsanitized into the SQL query.

3. Context

When a user selects a product category, the application executes the following query:

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

4. Exploitation Steps

1. Identify the category parameter in the URL (e.g., **Gifts**).
2. Modify the category parameter by injecting the payload below into the request.
3. Submit the request. The response now includes one or more unreleased products.

5. Payload

```
'+OR+1=1--
```

6. Technical Explanation

The injected payload breaks out of the string literal and appends **OR 1=1**, which always evaluates to true. The double-dash (**--**) comments out the remainder of the query, including the **AND released = 1** condition. As a result, the query returns all products regardless of their release status.

7. Result

The application returned unreleased products that were not intended to be visible to users.

Screenshot — Lab Solved

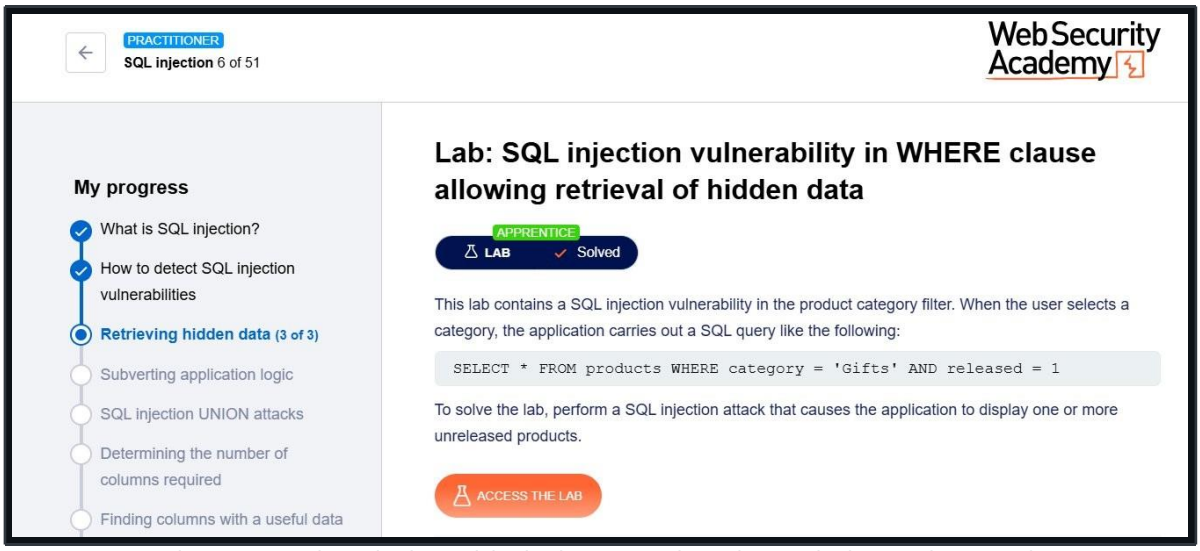

Figure 1.1 — Lab 1 solved: SQL injection in WHERE clause (Apprentice | PortSwigger WSA)

LAB 2

SQL Injection — Login Bypass

LAB 02	APPRENTICE	SOLVED
SQL Injection Vulnerability Allowing Login Bypass		

Difficulty: Apprentice	**Target:** Login function	**Status:** Solved

2. Vulnerability

The login function is vulnerable to SQL injection. The username field does not sanitize user input before incorporating it into the authentication query.

3. Context

The target is a web application login page hosted on the Web Security Academy platform.

4. Exploitation Steps

1. Navigate to the login page of the application.
2. In the username field, enter the payload below.
3. Submit the form. Authentication is bypassed and access is granted as the administrator.

5. Payload

```
' OR '1'='1--
```

6. Technical Explanation

The injected payload terminates the username string and appends a condition **OR '1'='1'**, which always evaluates to true. The double-dash comments out any remaining query logic, including the password check. The application authenticates the session without valid credentials.

7. Result

Authentication was bypassed. Access was granted to the administrator account without providing a valid password.

Screenshots — Lab Description and Solved State

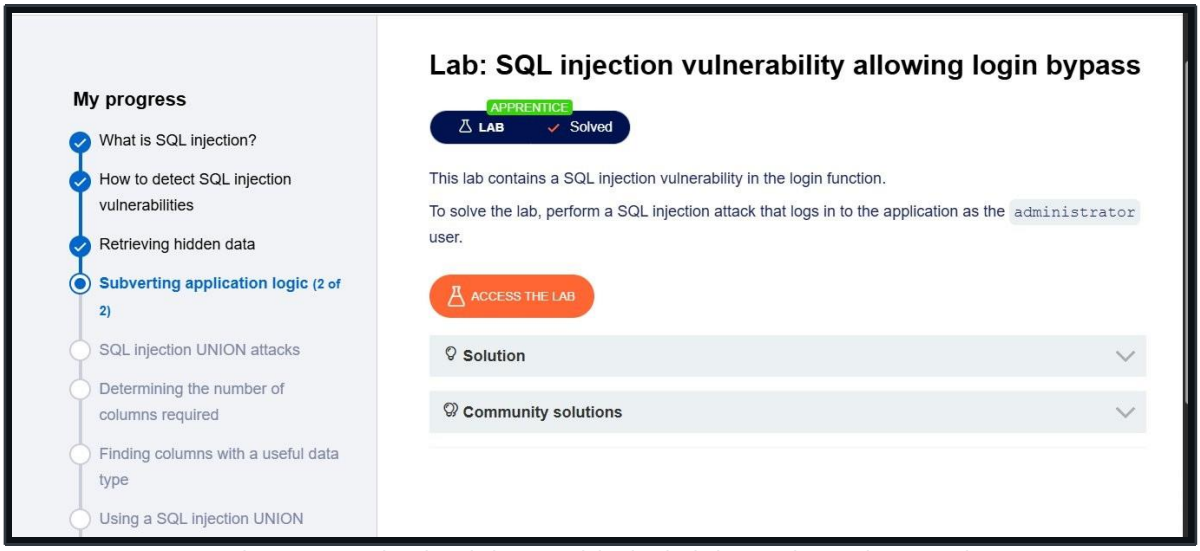

Figure 2.1 — Lab 2 description: SQL injection login bypass (PortSwigger WSA)

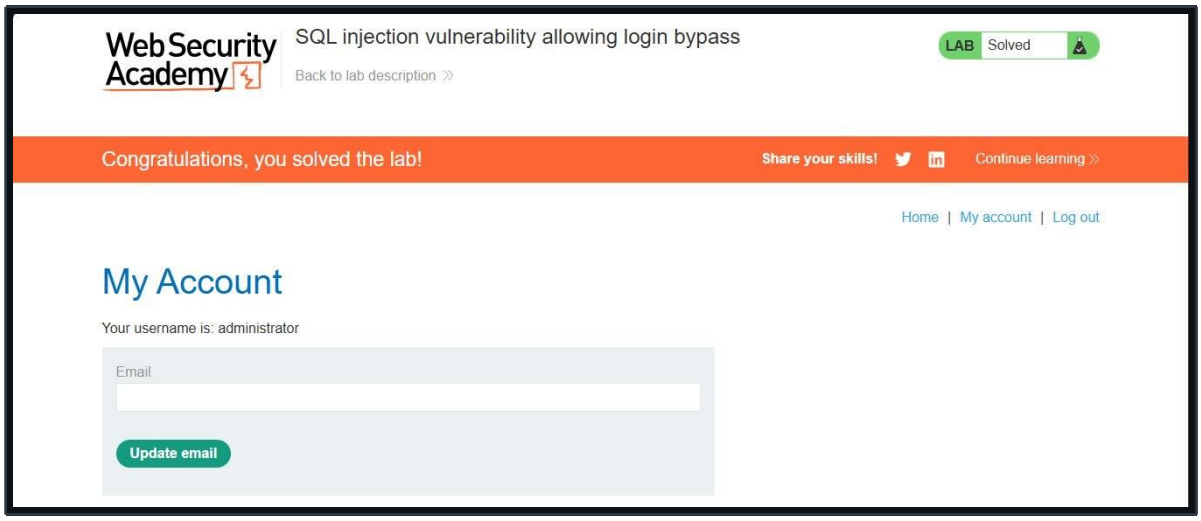

Figure 2.2 — Lab 2 solved: authenticated as administrator

LAB 3

SQL Injection UNION Attack — Determining Column Count

LAB 03	PRACTITIONER	SOLVED
SQL Injection UNION Attack: Column Count		

Difficulty: Practitioner	**Target:** Product category filter	**Status:** Solved

2. Vulnerability

The product category filter is vulnerable to SQL injection. Query results are reflected in the application response, enabling UNION-based attacks.

3. Context

The objective is to determine the number of columns returned by the original query by injecting UNION SELECT statements with NULL values.

4. Exploitation Steps

1. Inject **' UNION SELECT NULL--** to test for a single-column query.
2. If an error is returned, increase the number of NULLs by one: **' UNION SELECT NULL,NULL--**
3. Continue adding NULLs until no error is returned. The count of NULLs at that point equals the number of columns in the original query.

5. Payloads

```
' UNION SELECT NULL--

' UNION SELECT NULL,NULL--

filter?category=Gifts%27+UNION+SELECT+NULL,+NULL,+NULL--

?category=Gifts' UNION SELECT NULL,'b',NULL--
```

6. Technical Explanation

A UNION SELECT must return the same number of columns as the original query. By incrementally adding NULL placeholders, it is possible to identify the exact column count. The absence of an error confirms a match. The working payload with the correct number of NULLs reveals the column count of the original query.

7. Result

The exact number of columns returned by the original query was determined.

Screenshot — Lab Solved

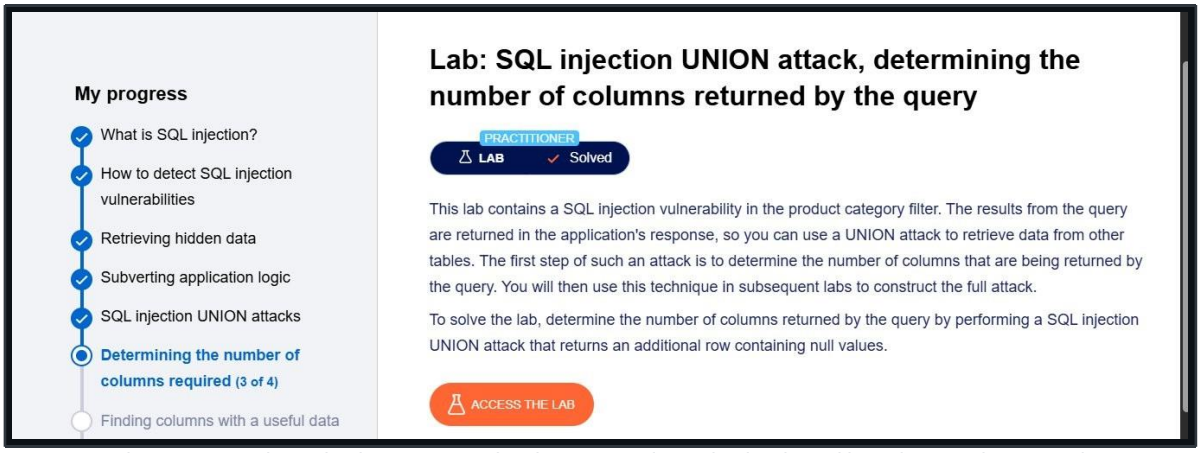

Figure 3.1 — Lab 3 solved: UNION attack column count determination (Practitioner | PortSwigger WSA)

LAB 4

SQL Injection UNION Attack — Finding a Column Containing Text

LAB 04	PRACTITIONER	SOLVED
SQL Injection UNION Attack: Column with Text		

Difficulty: Practitioner	**Target:** Product category filter	**Status:** Solved

2. Vulnerability

Following column count determination, the application is further exploited to identify which specific column renders string data in the response.

3. Context

The lab provides a random string value that must appear in the query results. The goal is to identify which column is compatible with string data.

4. Exploitation Steps

1. Using the column count from Lab 3, replace one NULL at a time with a string value.
2. If the string appears in the page response, that column accepts and displays string data.
3. The string **'Ri6fJY'** appeared in the response when placed in the second column position, confirming the second column as the target.

5. Payload

```
?category=Gifts%27+UNION+SELECT+NULL,%27Ri6fJY%27,NULL--
```

6. Technical Explanation

Not all columns in a query necessarily return data that is rendered on the page. By substituting a known string into each NULL position one at a time, it is possible to identify which column outputs its value to the response. The appearance of **'Ri6fJY'** in the response confirmed the second column as the string-compatible column.

7. Result

The second column was identified as the column that renders string data in the application response.

Screenshot — Lab Solved

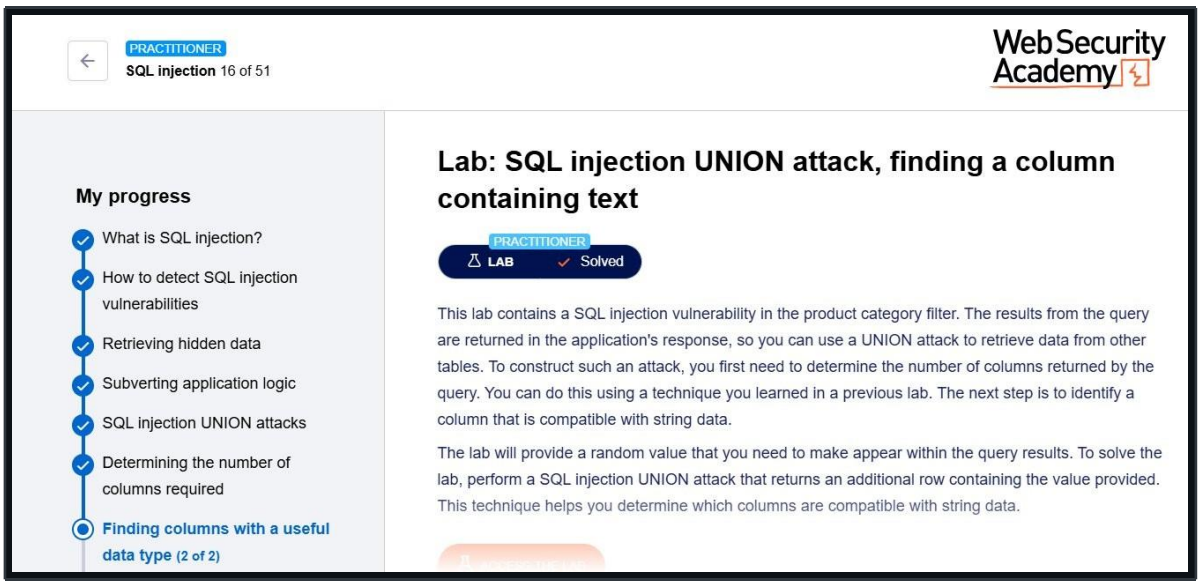

Figure 4.1 — Lab 4 solved: UNION attack finding text column (Practitioner | PortSwigger WSA)

LAB 5

SQL Injection UNION Attack — Retrieving Data from Other Tables

LAB 05	PRACTITIONER	SOLVED
SQL Injection UNION Attack: Data from Other Tables		

Difficulty: Practitioner	**Target:** Product category filter / users table	**Status:** Solved

2. Vulnerability

Combining the techniques from Labs 3 and 4, the SQL injection vulnerability is used to extract credentials from a separate database table.

3. Context

The database contains a table named **users** with columns **username** and **password**. The original query returns 3 columns. The second column renders string data on the page.

4. Exploitation Steps

1. Inject a UNION SELECT targeting the **users** table, mapping **username** and **password** to the correct column positions.
2. The second column (string-compatible) displays the administrator's password in the response.
3. Use the retrieved administrator credentials to log in and complete the lab.

5. Payload

```
'+UNION+SELECT+username,+password+FROM+users--
```

6. Technical Explanation

The payload appends a secondary SELECT statement via UNION, targeting the **users** table. The query matches the 3-column requirement established in Lab 3. The **username** and **password** values are mapped to the first two columns. Since the second column is the visible one (identified in Lab 4), the administrator's password is rendered in the application's response.

7. Result

The administrator's username and password were retrieved from the **users** table. These credentials were used to authenticate as the administrator.

Screenshot — Lab Solved

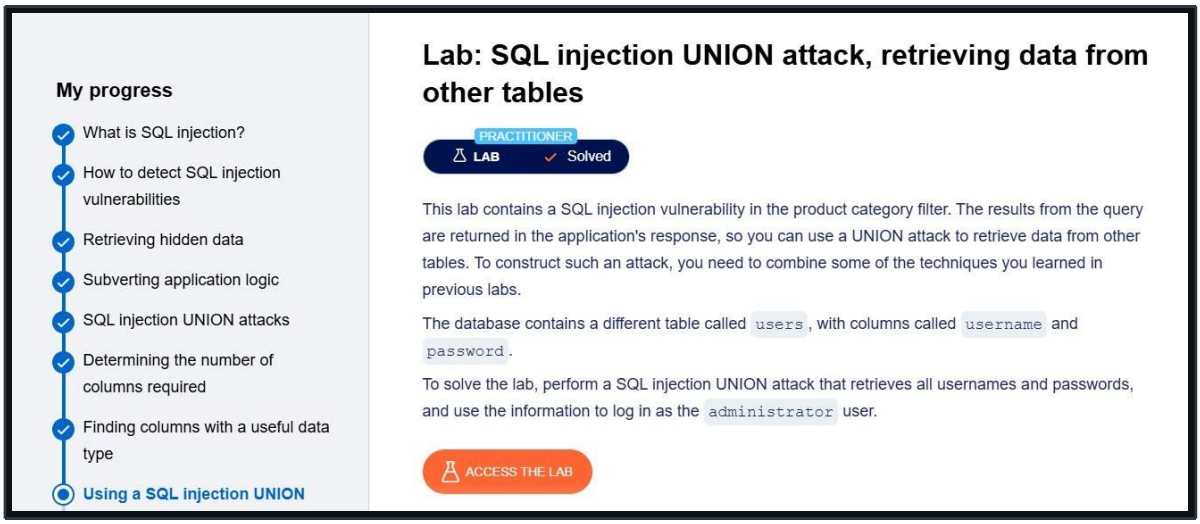

Figure 5.1 — Lab 5 solved: UNION attack retrieving data from other tables (Practitioner | PortSwigger WSA)

Impact & Mitigation

Impact

Lab	Impact
1 — Hidden Data	Unreleased products disclosed to unauthenticated users
2 — Login Bypass	Full authentication bypass; administrator account access without credentials
3 — Column Count	Internal query structure revealed; enables further UNION-based exploitation
4 — Text Column	Identifies which column renders output; required for data extraction
5 — User Data	Full credential dump from the users table; administrator login achieved

Mitigation

Use Parameterized Queries	All database queries must use parameterized statements or prepared statements. User input must never be concatenated directly into SQL strings.
Input Validation	Validate and sanitize all user-supplied input on the server side before any processing.
Least Privilege	Database accounts used by the application should have the minimum permissions required. They must not have access to sensitive tables such as user credentials.
Error Handling	Suppress detailed database error messages from being returned to the client. Log errors server-side only.

End of Report — All 5 labs completed and solved on the Web Security Academy platform.